

SE 811: Software Maintenance

Toukir Ahammed

Perfective Change

- Perfective change is the change undertaken to expand the existing requirements of a system.
- When a software product becomes useful, users always expect to use it in new scenarios beyond the scope for which it was initially developed.
- Such requirement expansion causes changes to either enhance existing system functionality or to add new features.
- For instance, an image processing system is originally developed to process JPEG files and later goes through a series of perfective changes to handle other formats, such as PNG and SVG.

Tyranny of Dominant Decomposition

- As programs evolve over time, they may suffer from the *tyranny of dominant decomposition*.
- They can be modularized in only one way at a time. Concerns that are added later may end up being scattered across many modules and thus tangled with one another.

Crosscutting concerns

- Crosscutting concerns refer to the *secondary design decisions* such as logging, performance, error handling, and synchronization.
- Adding these secondary concerns often involves nonlocalized changes throughout the system, due to the *tyranny* of dominant design decisions already implemented in the system.

Language Support for Crosscutting Concerns

Aspect-oriented programming (AOP)

- *Aspect-oriented programming* (AOP) is a programming paradigm that aims to increase modularity by allowing the separation of crosscutting concerns.
- Suppose developers want to add a new feature such as logging to log all executed functions.
- The logging logic is straightforward: printing the function's name at each function's entry.
- However, manually inserting the same implementation to each function body is tedious and error-prone.

Language Support for Crosscutting Concerns

Aspect-oriented programming (AOP)

- With AOP, developers only need to first define the logging logic as **an advice** and then specify the place where to insert the advice (i.e., **pointcut**), such as the entry point of each function.
- An aspect weaver will read the aspect-oriented code and generate appropriate object-oriented code with the aspects integrated.
- In this way, AOP facilitates developers to efficiently introduce new program behaviors without cluttering the core implementation in the existing codebase.

Language Support for Crosscutting Concerns

Aspect-oriented programming (AOP)

- Many Java bytecode manipulation frameworks implement the AOP paradigm, like ASM, Javassist , and AspectJ , so that developers can easily modify program runtime behaviors without touching source code.
- The benefit of AOP during software evolution is that crosscutting concerns can be contained as a separate module such as an aspect with its pointcut and advice description and thus reduces the developer effort in locating and updating all code fragments relevant to a particular secondary design decision such as logging, synchronization, database transaction, etc.